



RNN

Jan Polanco Velasco

RNN

- ◊ “El gato se comió el ___”.
- ◊ ¿Cómo sabe la red que sigue “pescado” y no “avión”?
- ◊ Porque el orden importa.
- ◊ Las redes densas ven las palabras como un “saco de palabras” (bag of words), perdiendo la estructura temporal.

RNN

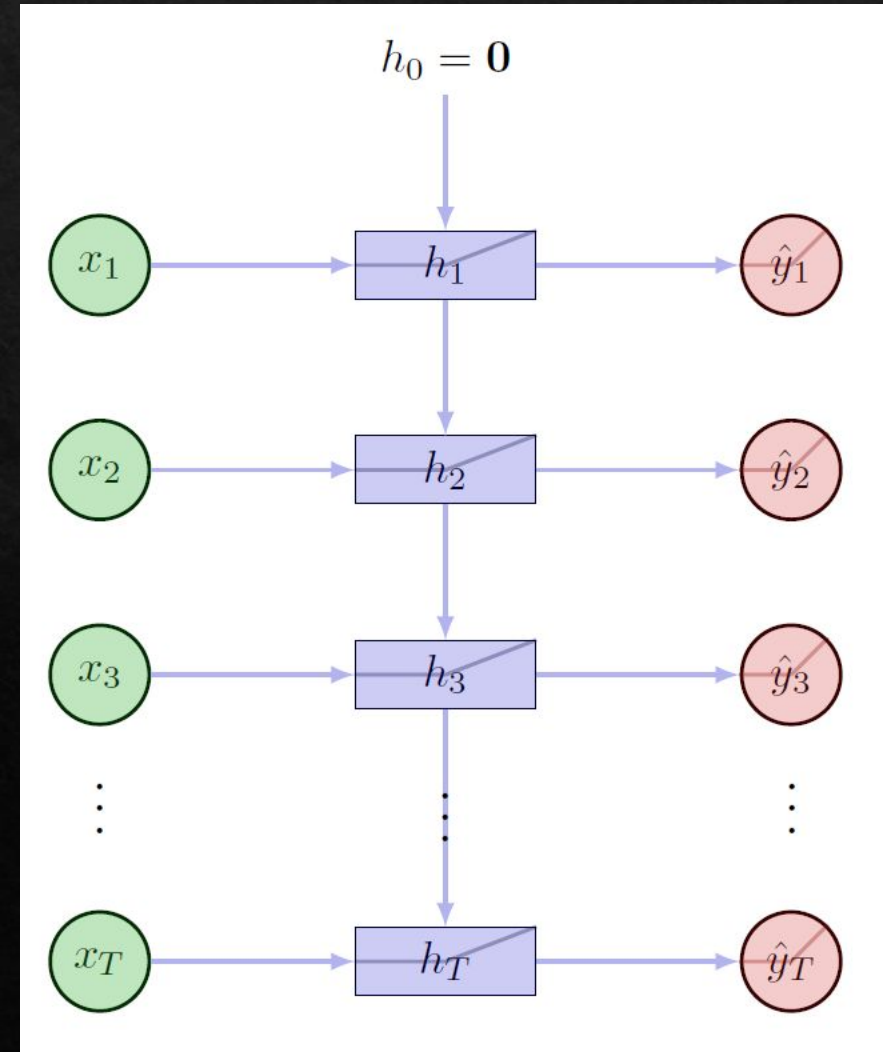
- ◊ RNN Recurrent Neural Network
- ◊ Las Fully Connected asumen independencia de las entradas.
- ◊ El problema de las RNN son las secuencias extensas donde se necesitan muchas unidades
- ◊ Se pierde la relación entre los elementos
- ◊ Sufre de gradientes que se desvanecen.

RNN

- ◆ SE USAN PARA:
- ◆ Reconocimiento de audio.
- ◆ Traducción automática.
- ◆ Generación de música
- ◆ Clasificación de sentimientos
(sentiment analysis)

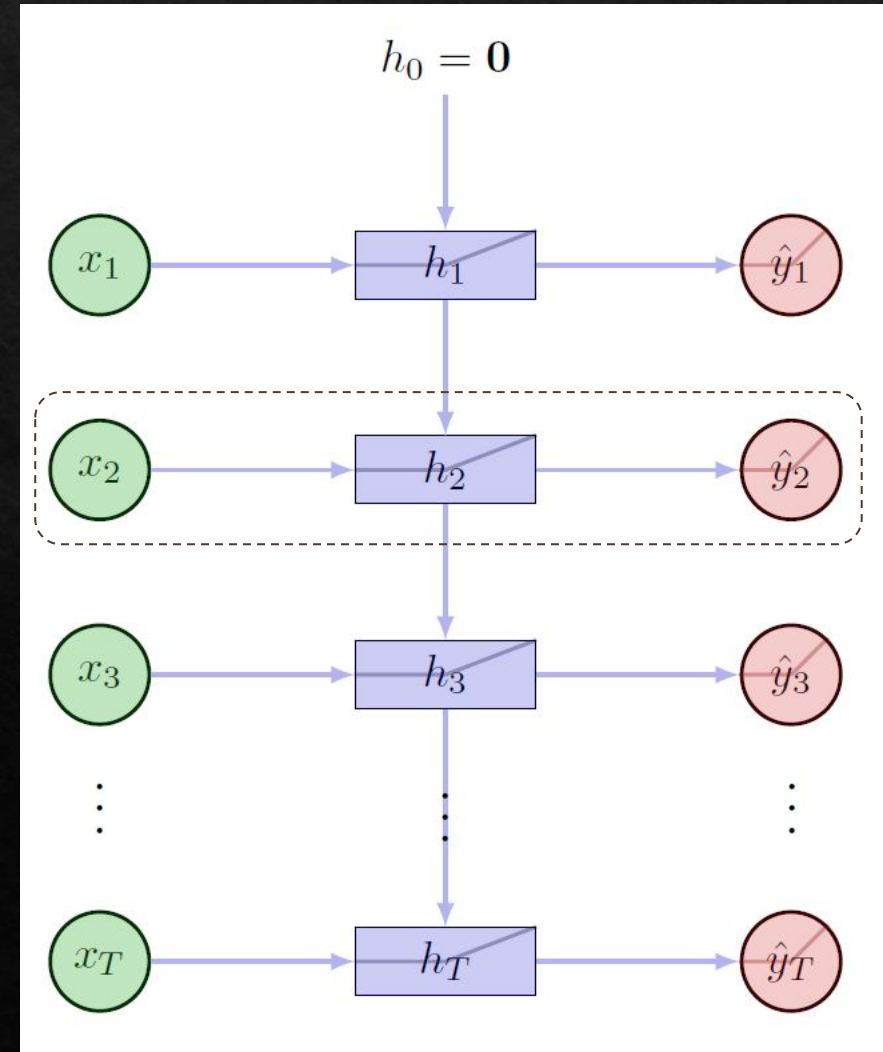
RNN

- ◊ Cada entrada x_j puede ser:
- ◊ Una representación numérica de una palabra
- ◊ Un fragmento de una señal de audio
- ◊ Un fragmento de una serie de tiempo, etc.
- ◊ Entity recognition “Many to Many”
- ◊ $h_1, h_2, \dots, h_T$ son el estado Oculto (Hidden State), memoria
- ◊ $h_0 = 0$ no hay recuerdos



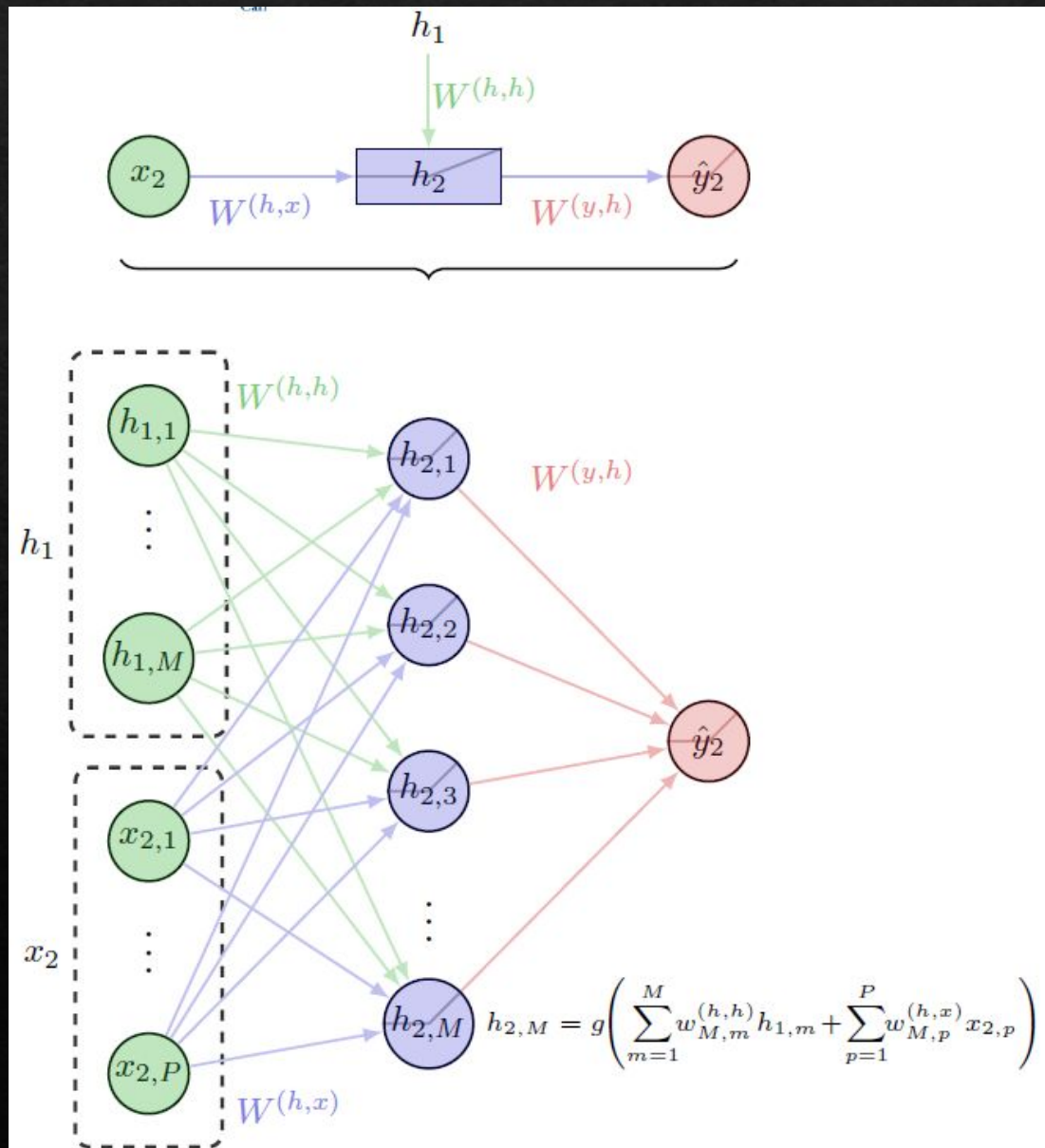
RNN

- ◊ La salida actual, depende de todas las entradas anteriores.
- ◊ Ejemplo el segundo elemento, con salida $\hat{y}_2$. Depende de todas las entradas anteriores.
- ◊ Esto hace que preserve su estructura secuencial de los datos.



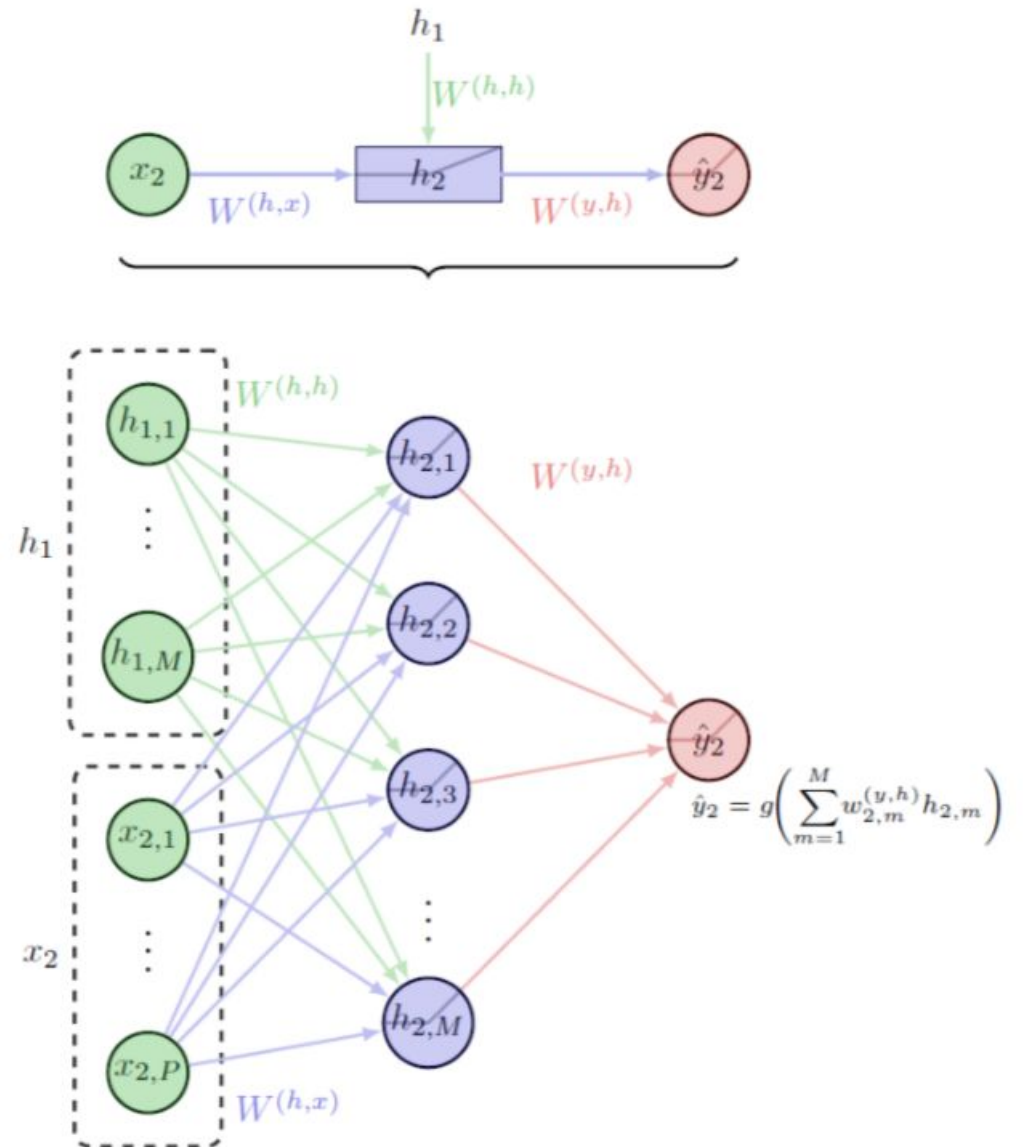
RNN

- ◊ Hacemos zoom
- ◊ Perspectiva de una fully connected.
- ◊ 3 Parámetros importantes (W)
- ◊ g es la función de activación
- ◊ h hidden layer.
- ◊ ejemplo $h_{2,M}$.
- ◊ La salida es $W(y,h) \times h_{2,m}$



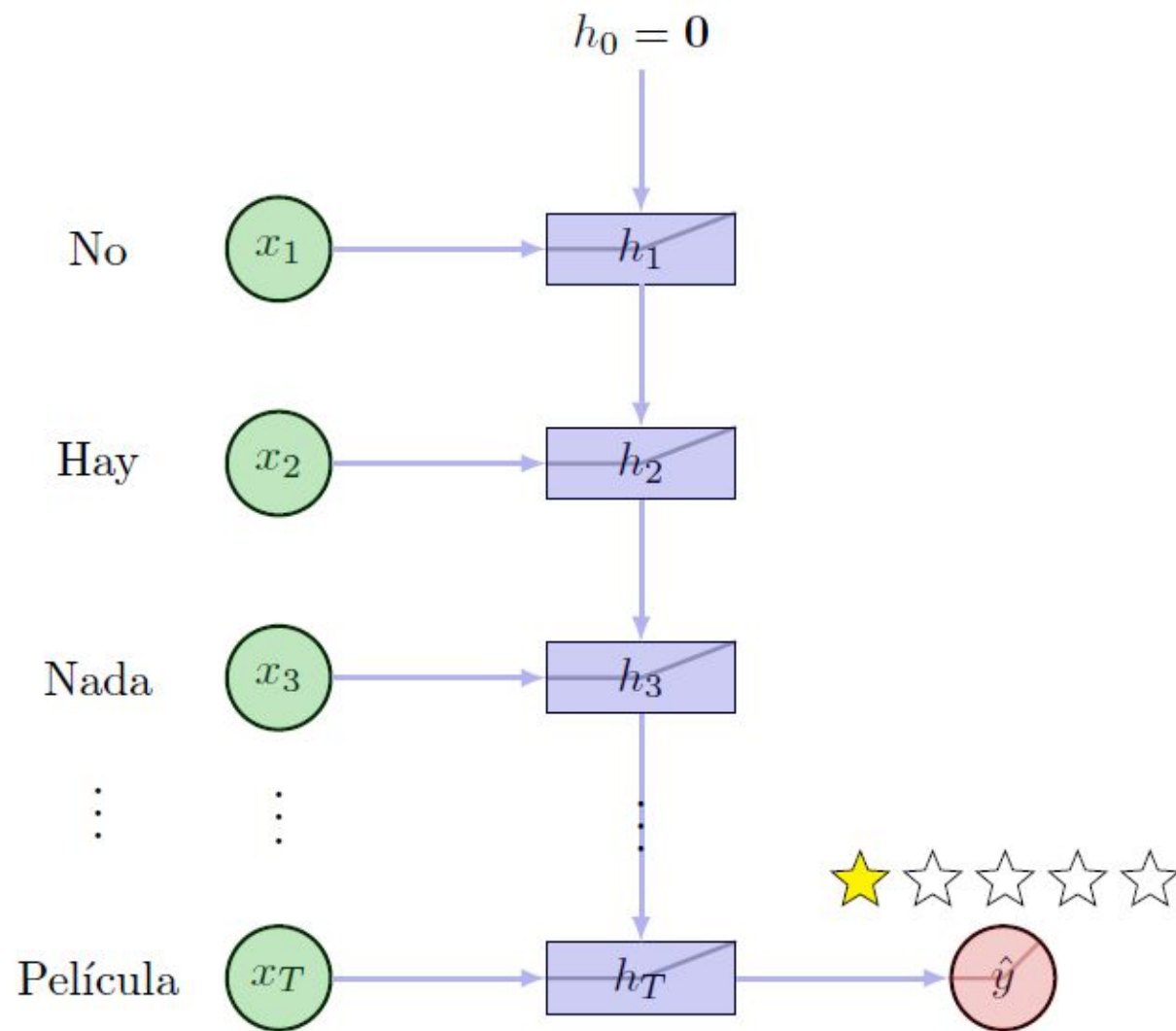
RNN

- ◆ Perspectiva de una fully connected.
- ◆ 3 Parámetros importantes (W)
- ◆ g es la función de activación
- ◆ h hidden layer.
- ◆ ejemplo $h_{2,M}$.
- ◆ La salida es $W(y,h) \times h_{2,m}$



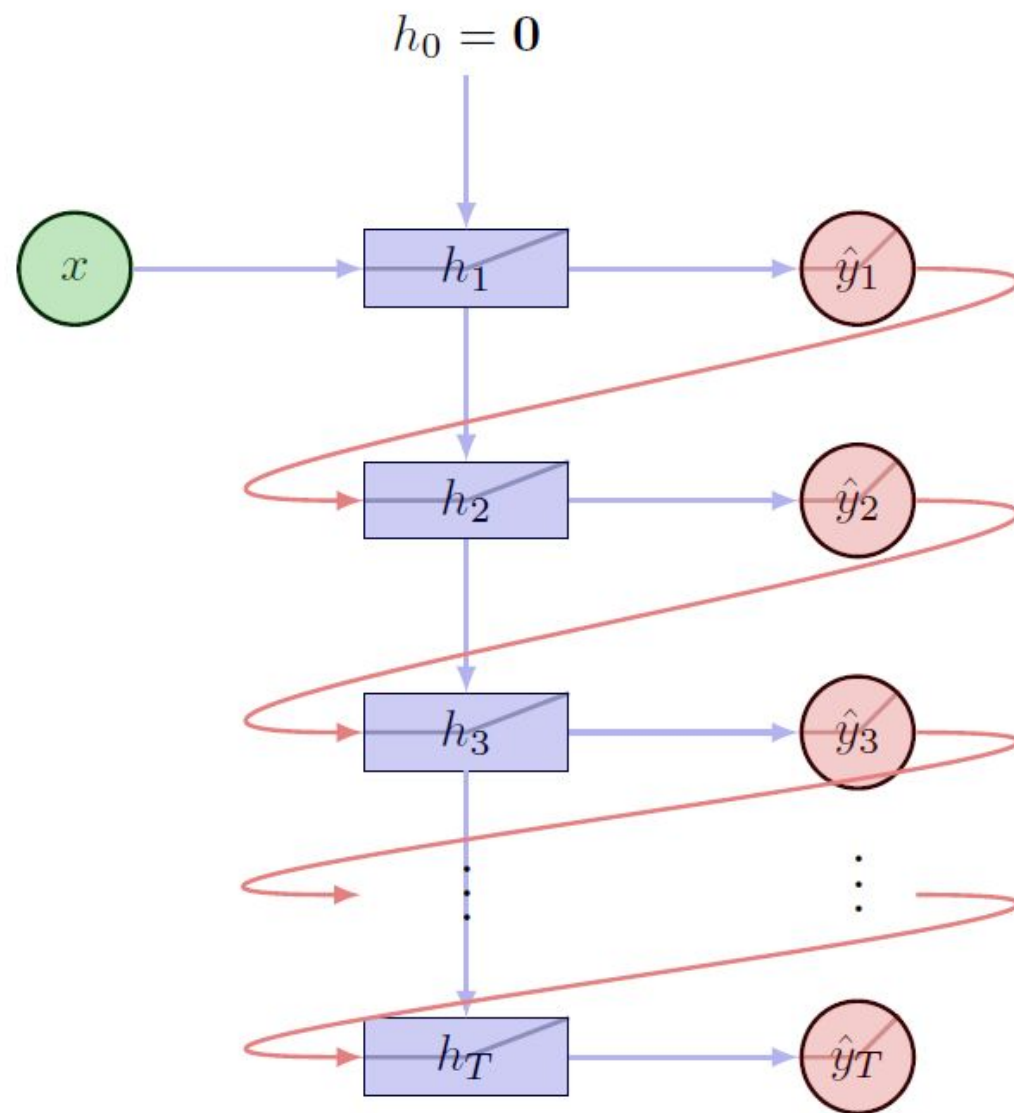
Variaciones de RNN

- ◊ Hasta el momento tenemos # Entradas = # Salidas.
- ◊ Arquitectura seria “Many to One”.
- ◊ Sentiment Analysis
- ◊ Entrada es una secuencia de texto
- ◊ Salida es una categoría.



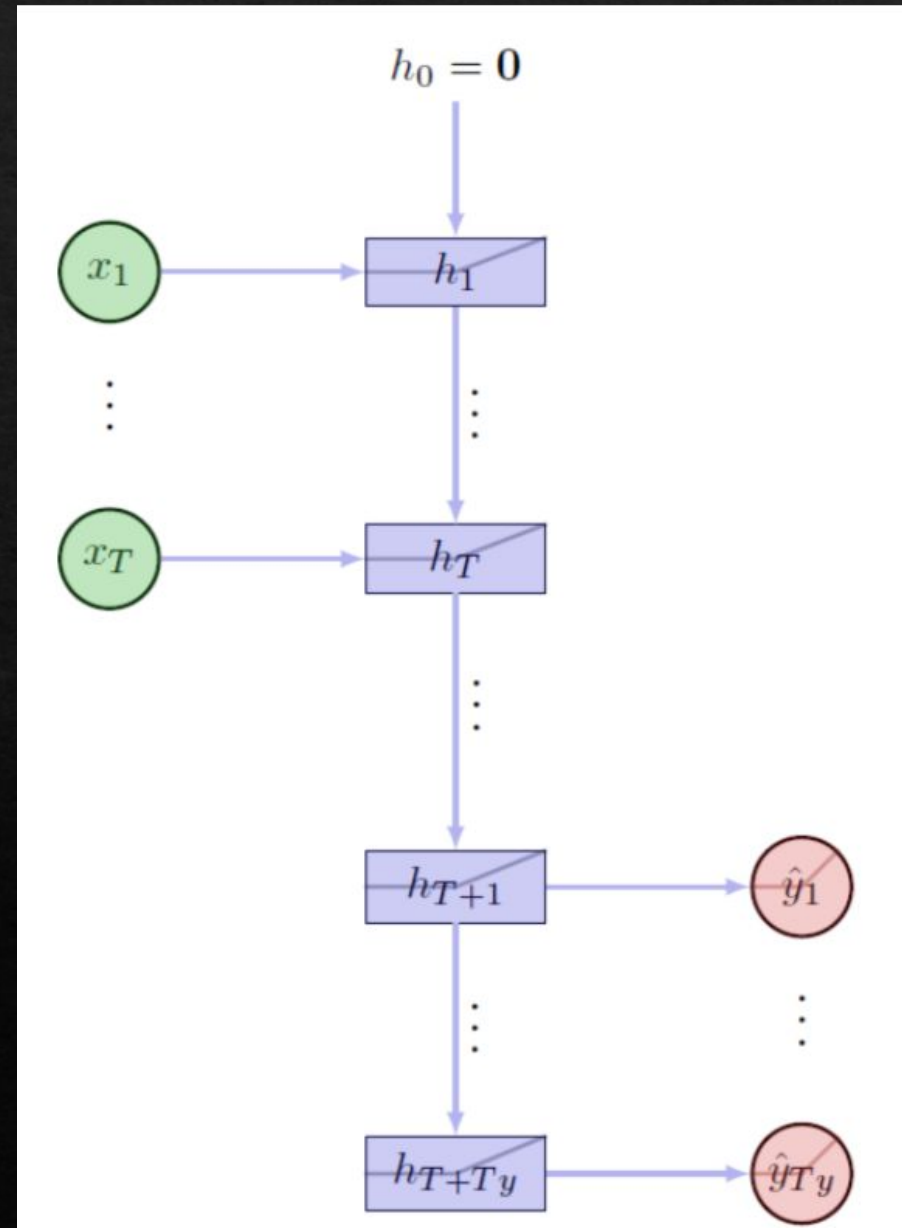
Variaciones de RNN

- ♦ Para generación de música
- ♦ 1 entrada (Un número)
- ♦ La salida es una secuencia.
- ♦ La secuencia puede ser una composición musical generada.
- ♦ “One to Many”
- ♦ La salida anterior es el input de la siguiente.

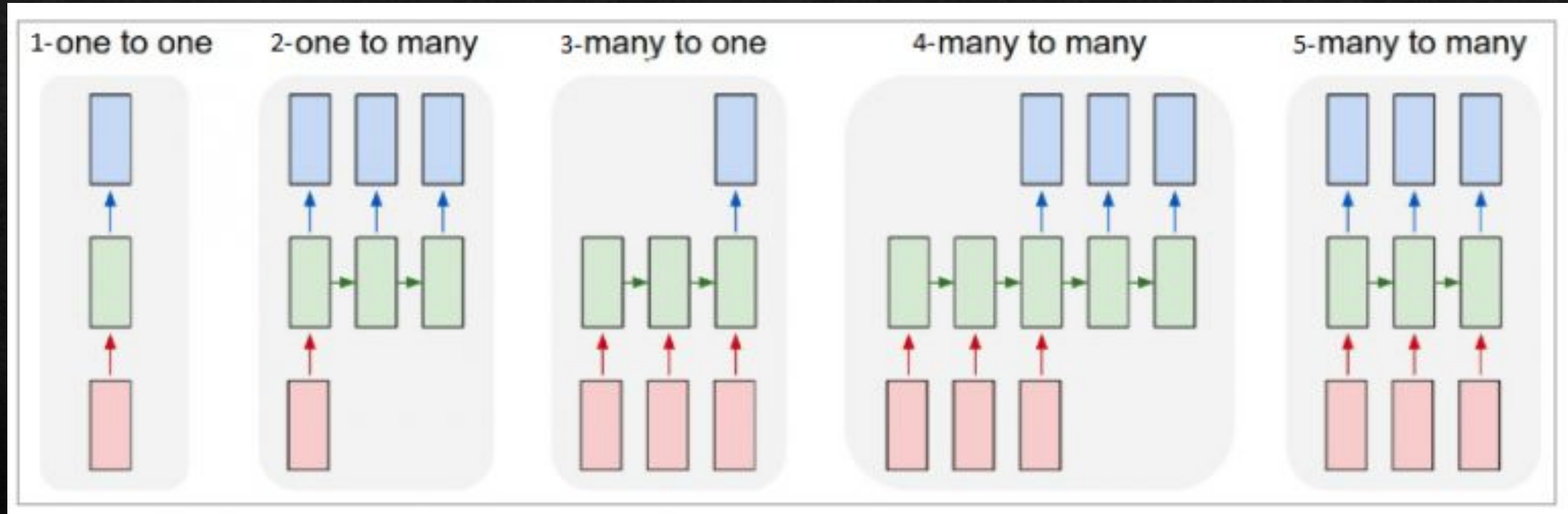


Variaciones de RNN

- ◊ Variación de “One to Many”.
- ◊ Entrada longitud
- ◊ Salida de otra longitud.
- ◊ Sistemas de traducción automática.

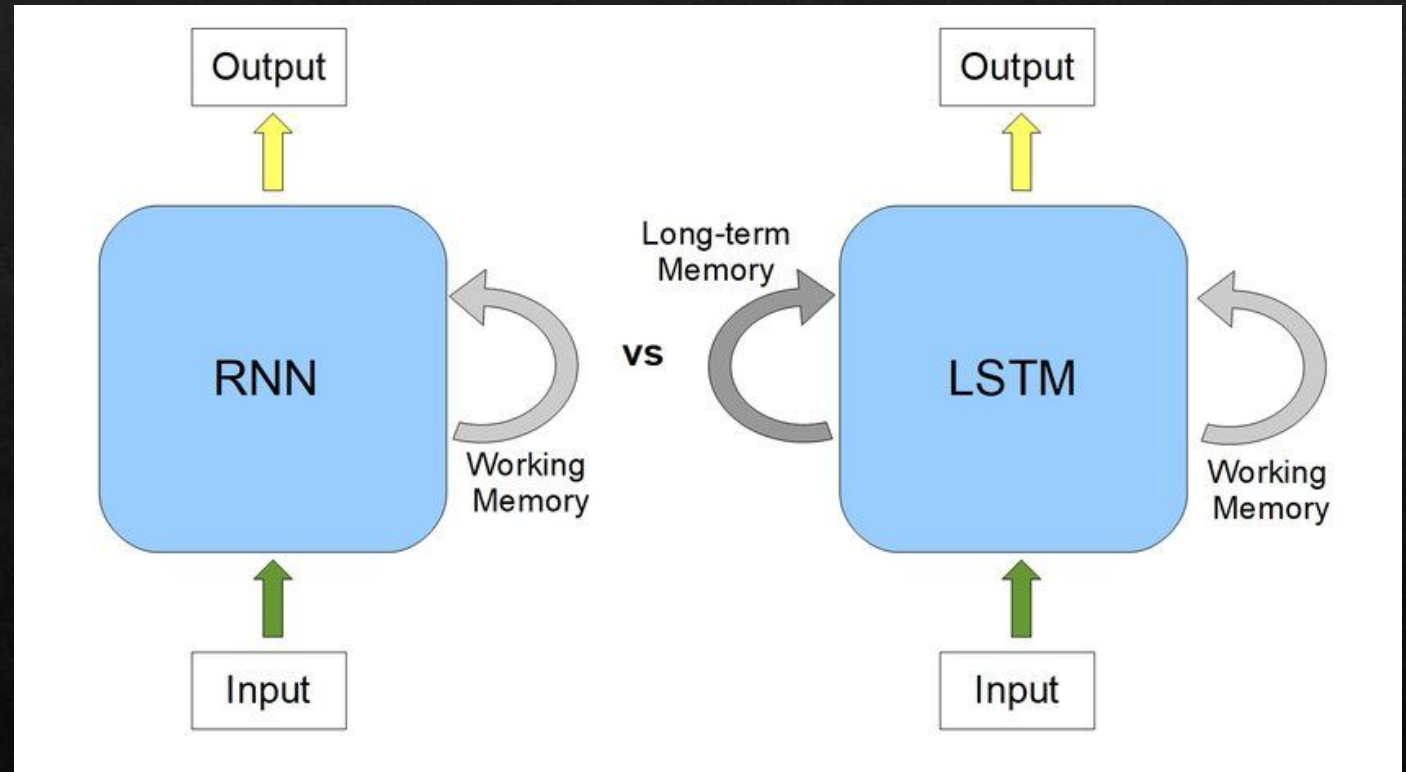


Variaciones de RNN



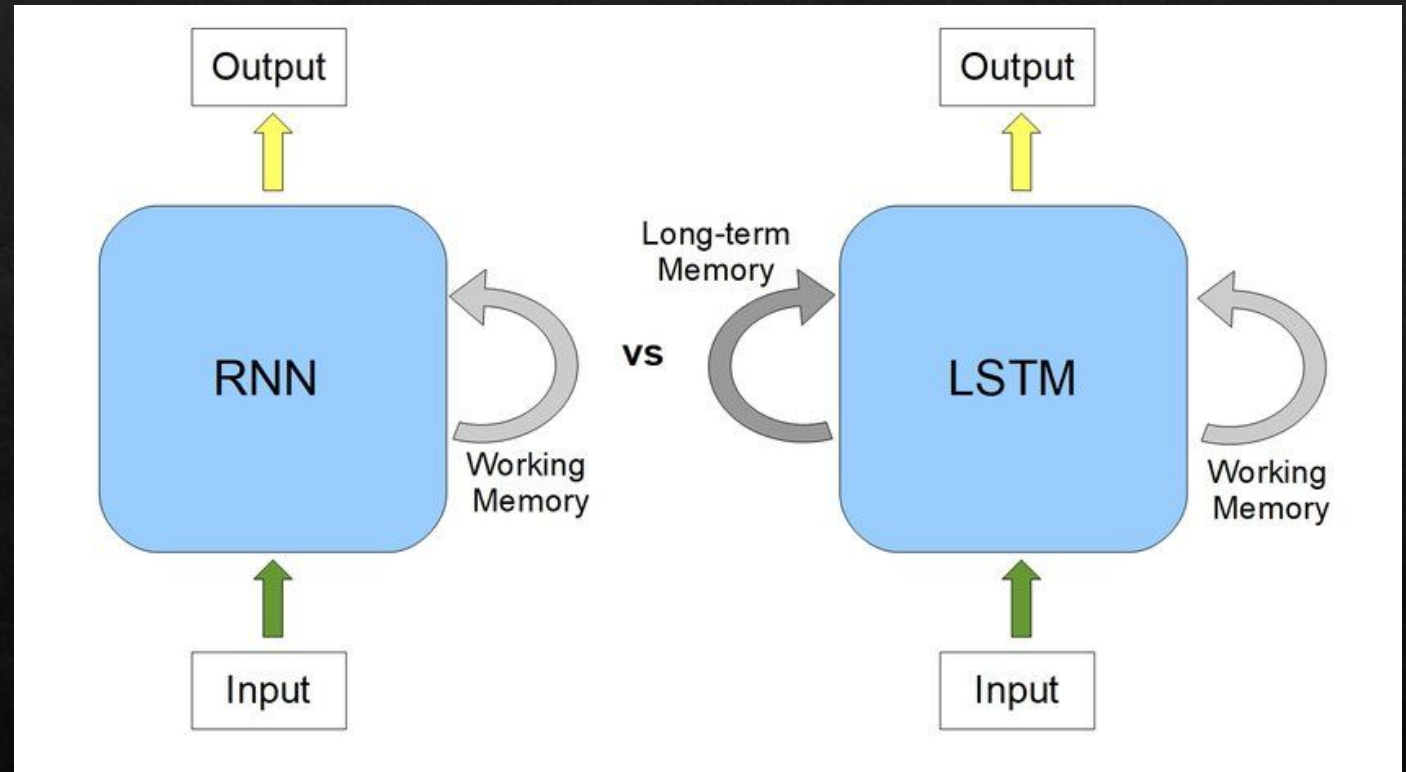
RNN con LSTM

- ♦ LONG SHORT TERM MEMORY
- ♦ Memoria a Largo y Corto Plazo” (LSTM, por sus siglas en inglés)
- ♦ Problemas con Backpropagation



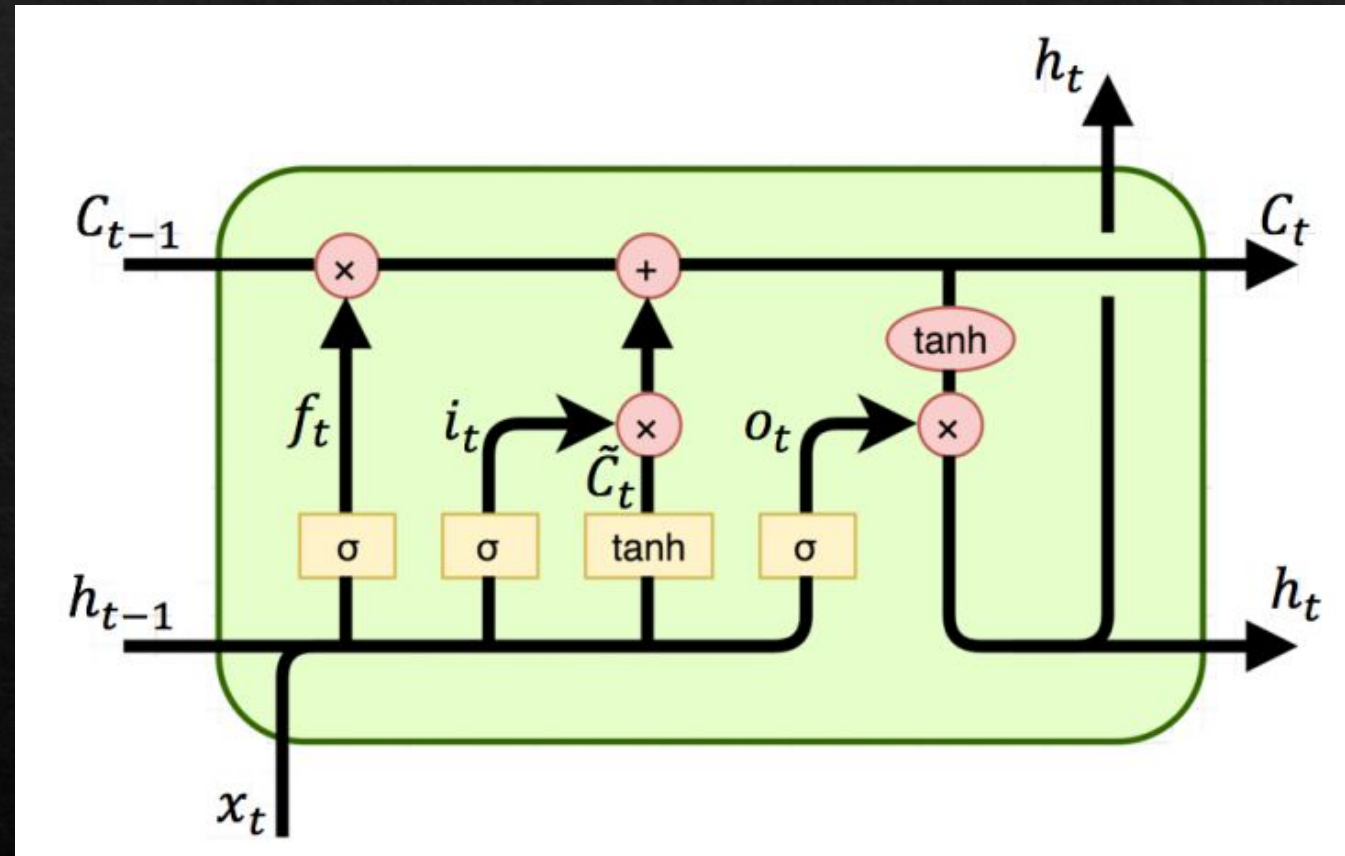
RNN con LSTM

- ◊ Computacionalmente eficiente.
- ◊ Una solución propuesta por Hochreiter y Schmidhuber en 1997.
- ◊ Evita el desvanecimiento del gradiente con celdas de memoria.



RNN con LSTM

- ♦ La Celda de Memoria: El corazón del LSTM
- ♦ Componentes principales:
- ♦ Puerta de entrada: Decide qué información entra a la celda.
- ♦ Puerta de olvido: Decide qué información descartar.
- ♦ Puerta de salida: Controla la salida hacia la siguiente celda.



RNN con LSTM

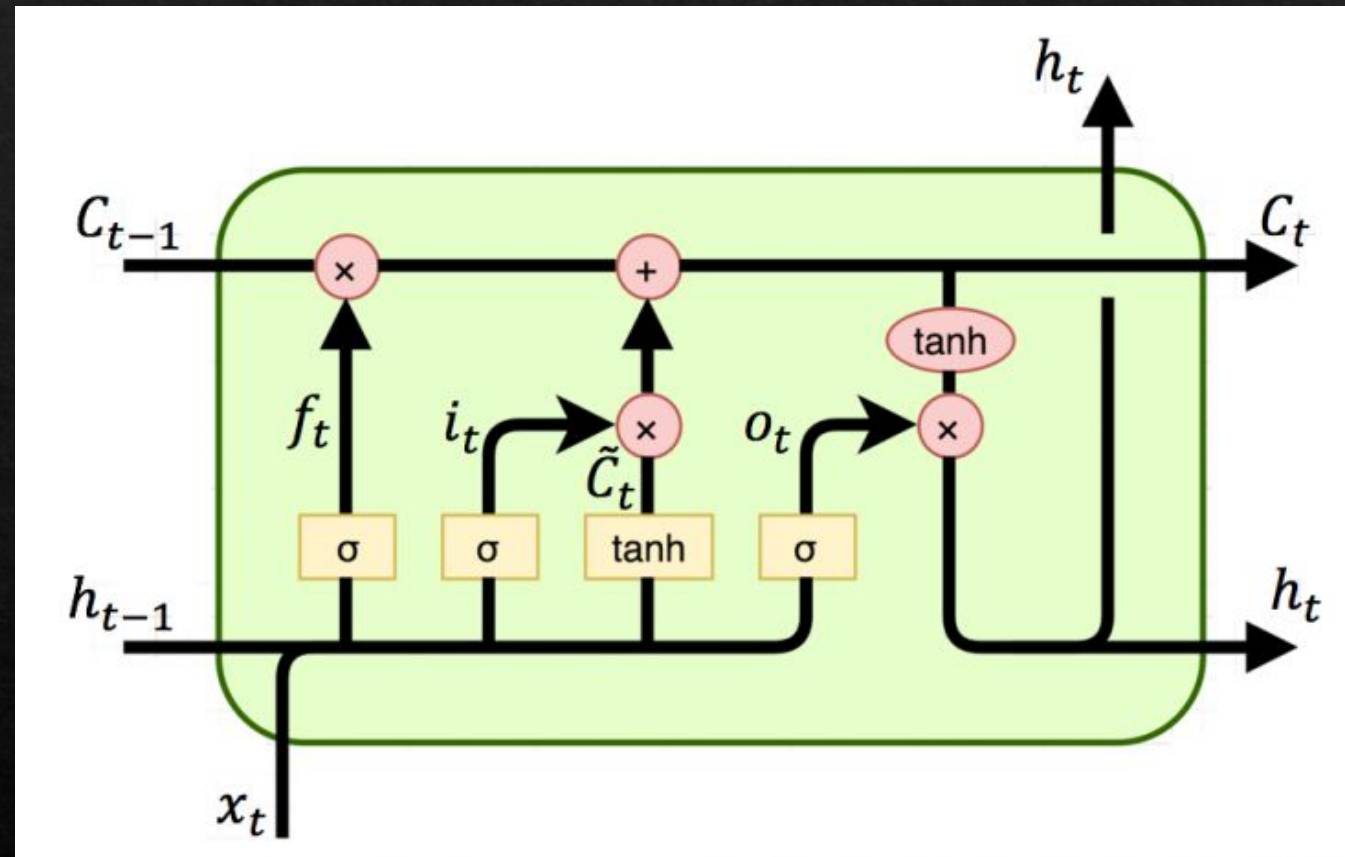
h_t, C_t : hidden layer vectors.

x_t : input vector.

b_f, b_i, b_c, b_o : bias vector.

W_f, W_i, W_c, W_o : parameter matrices.

$\sigma, \tanh$: activation functions.



RNN con LSTM

- Puerta de olvido:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- Puerta de entrada:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

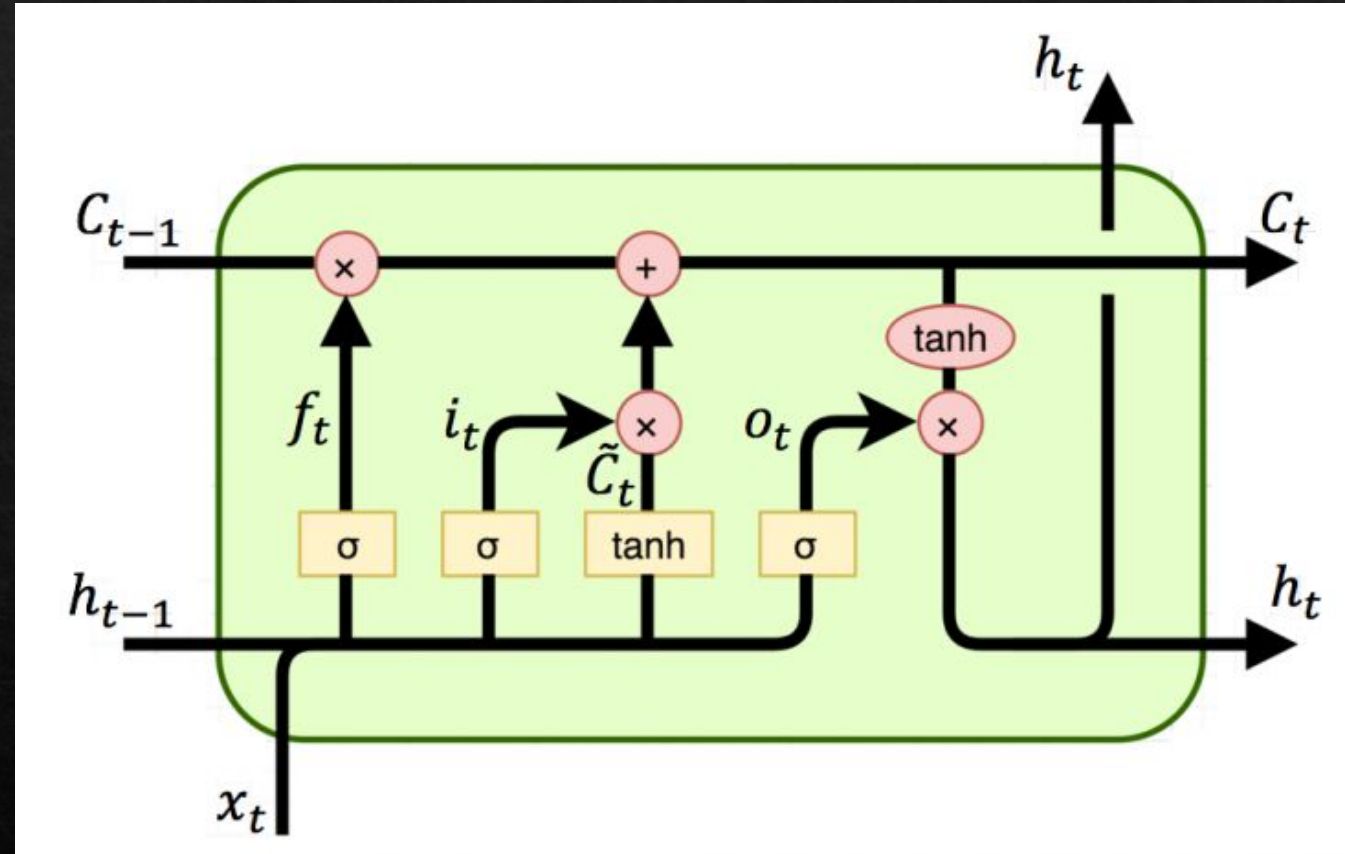
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

- Puerta de salida:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

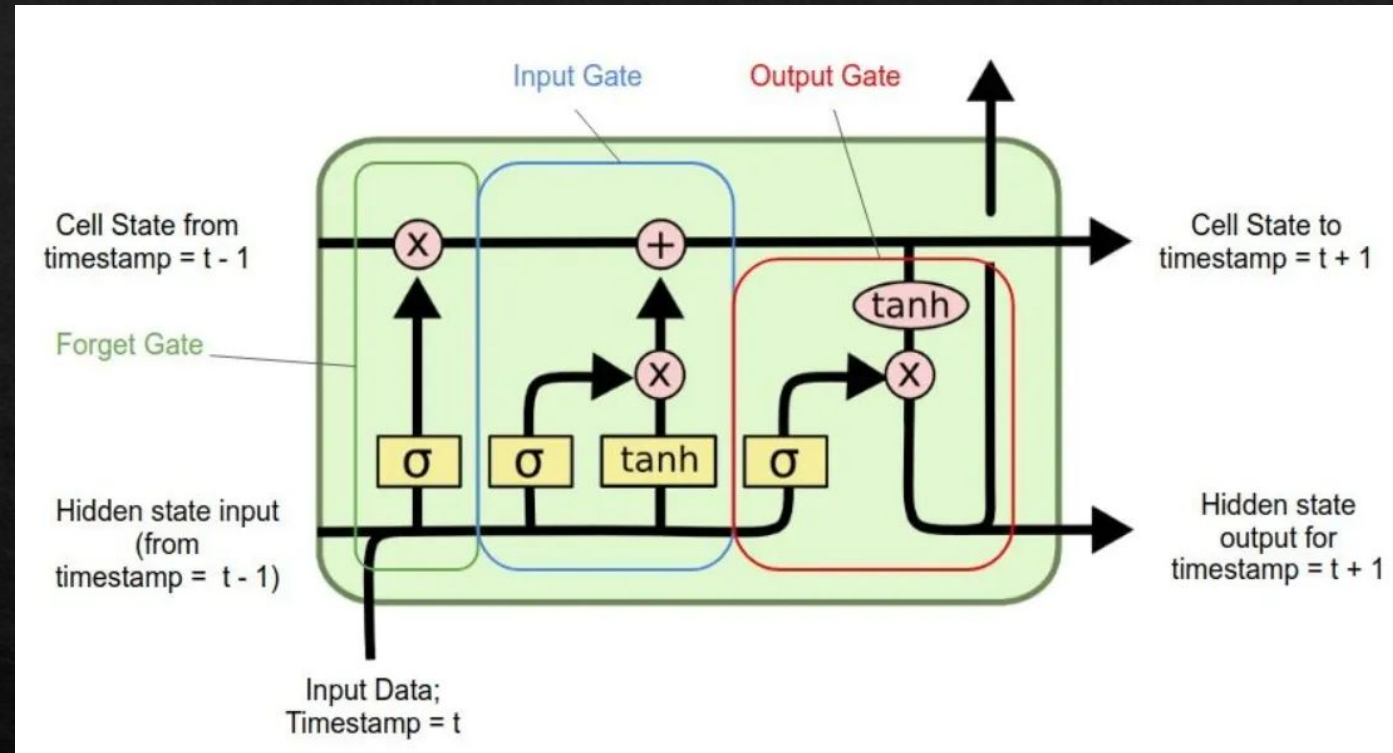
$$h_t = o_t \cdot \tanh(C_t)$$



¿Son las LSTM la solución definitiva?

Desventajas:

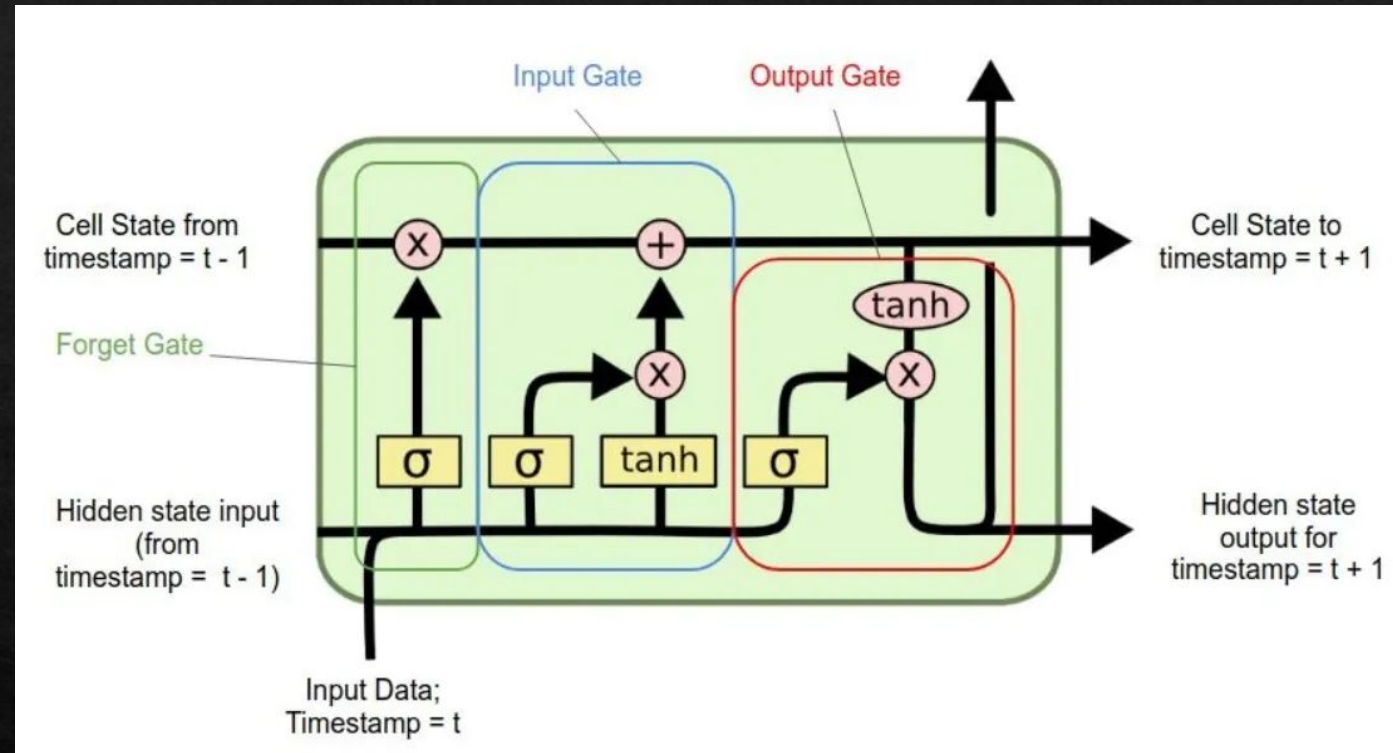
- ♦ Altos requerimientos computacionales.
- ♦ Problemas para manejar secuencias extremadamente largas.
- ♦ Más lentas en comparación con GRU o Transformers.
- ♦ Nota: Forget Gate - gradiente que se desvanece.



¿Son las LSTM la solución definitiva?

Alternativas modernas:

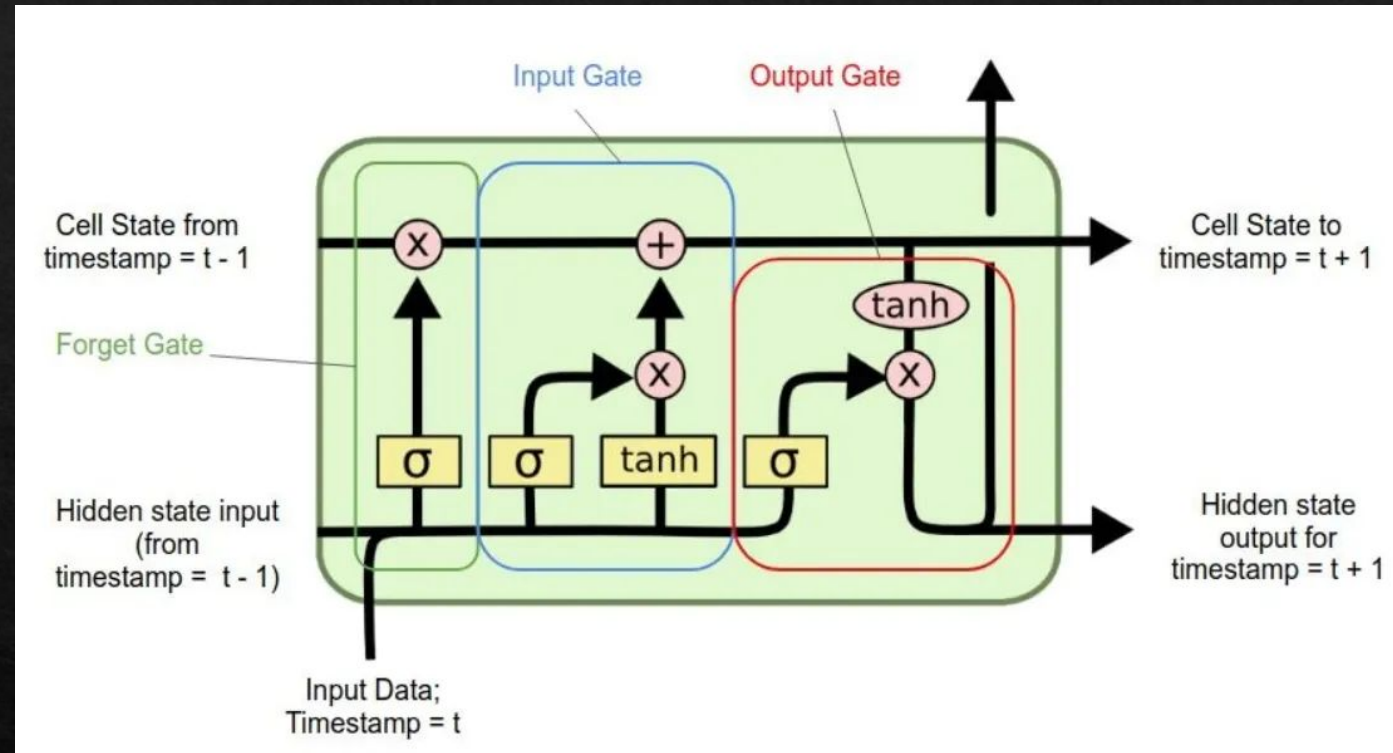
- ❖ GRU (Gated Recurrent Unit): Simplificación de LSTM.
- ❖ Transformers: Mejor rendimiento en tareas de secuencia larga.
- ❖ ¿Creen que las LSTM seguirán siendo relevantes frente a modelos como Transformers?



¿Son las LSTM la solución definitiva?

Ventajas de LSTM:

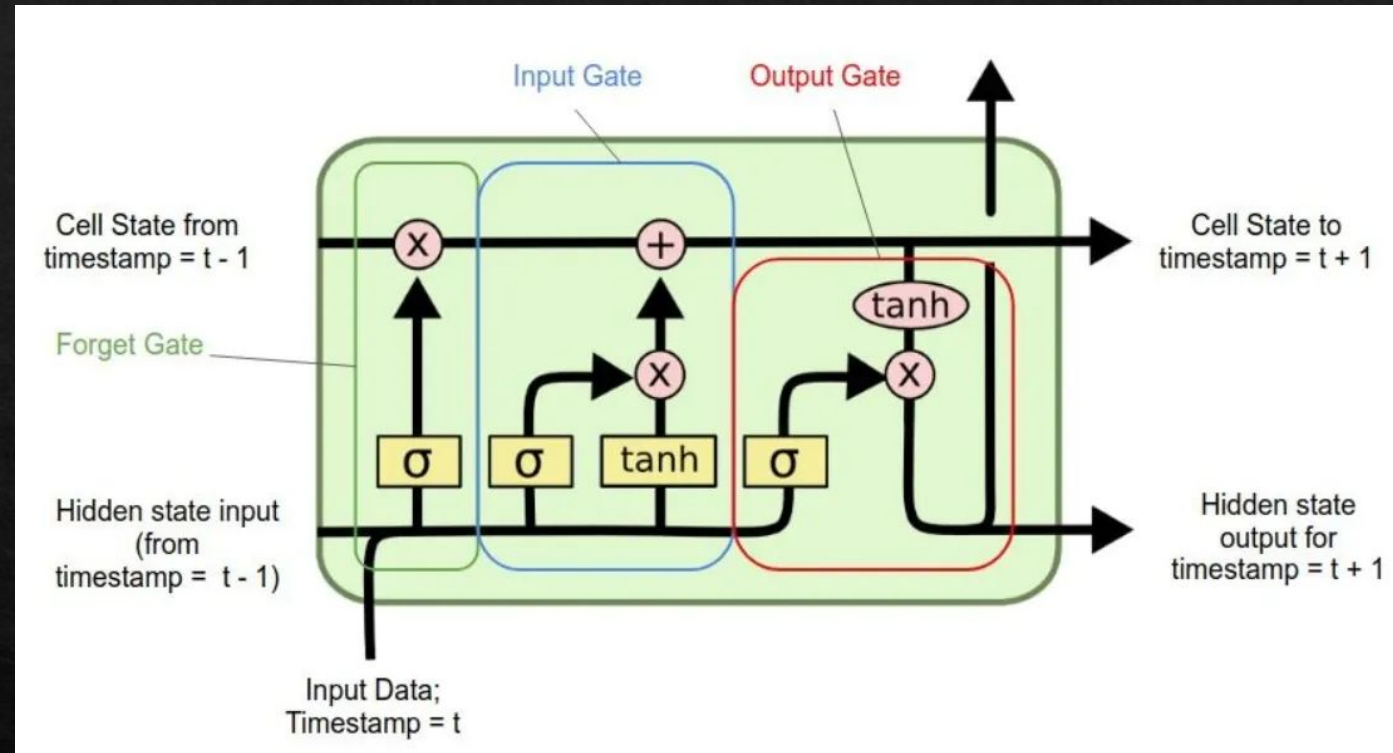
- ◆ Excelente manejo de secuencias con dependencias largas.
- ◆ Flexibilidad en múltiples dominios (NLP, series de tiempo, audio).



¿Son las LSTM la solución definitiva?

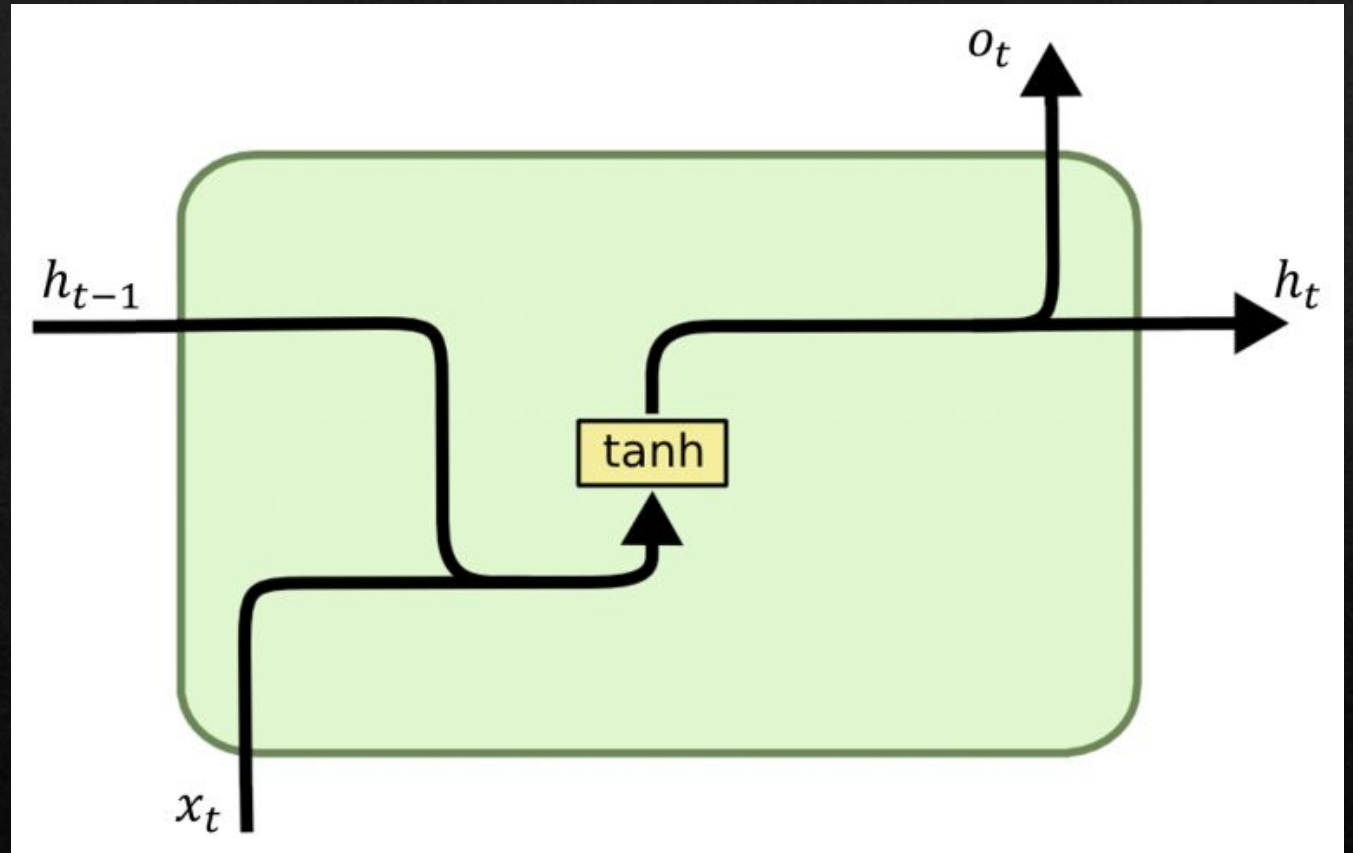
Desafíos actuales:

- ♦ Competencia con modelos más modernos.
- ♦ Optimización en hardware más accesible.
- ♦ Pregunta para el público:
- ♦ ¿Dónde aplicarían LSTM en su campo de estudio o trabajo?



RNN con GRU

- ♦ La imagen es una RRN
- ♦ Recordemos que es RNN tiene la capa oculta anterior.
- ♦ o_t es y_t .
- ♦ Se establece una compuerta para la capa oculta anterior.
- ♦ Permite decidir qué tanta información pasada servirá.



RNN con GRU

- ♦ La compuerta Gamma toma valores entre 0 y 1,

1. Entradas principales:

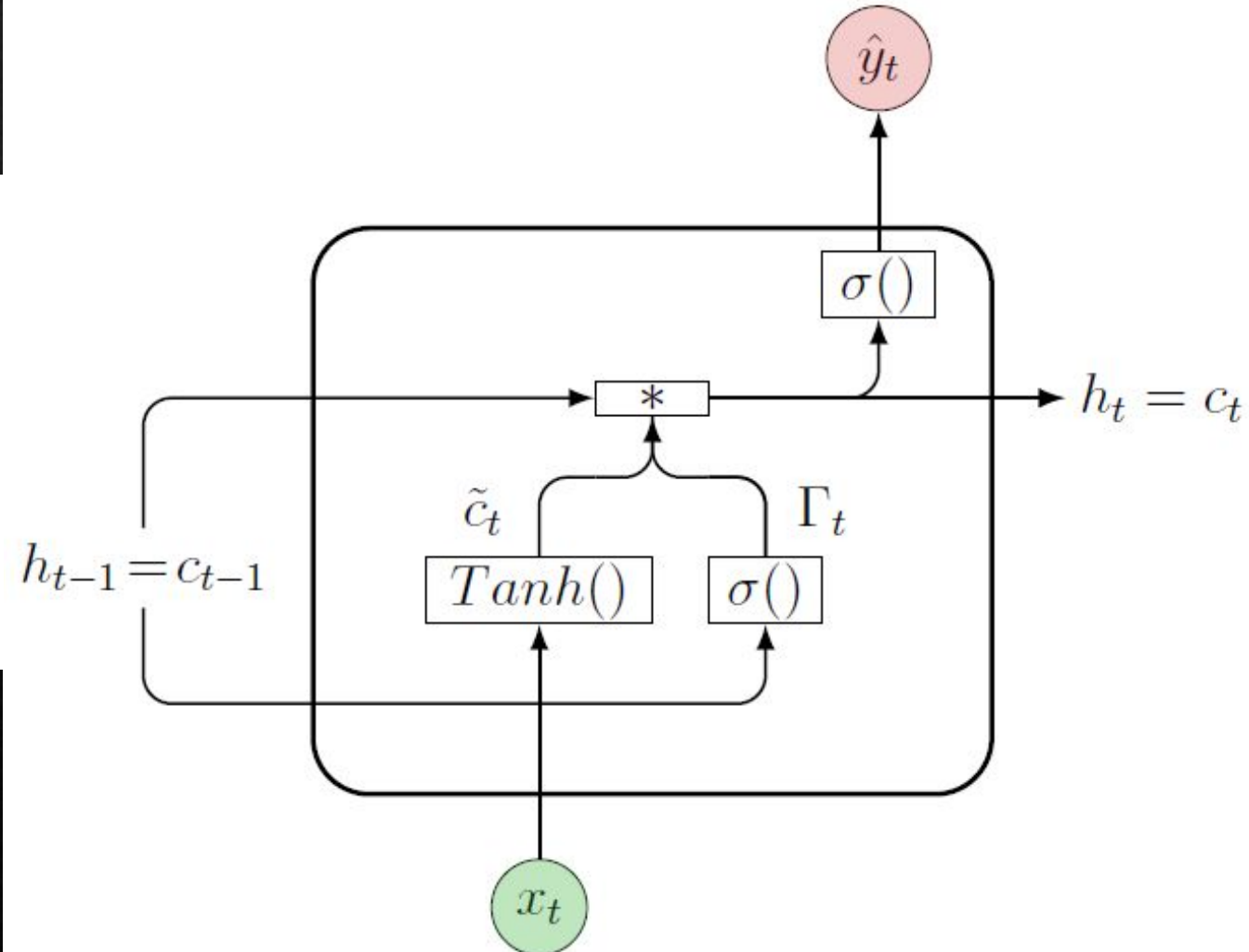
- $c_{t-1} = h_{t-1}$: Contiene información de unidades pasadas.
- x_t : Elemento actual de la secuencia.

2. Componente clave:

- $\tilde{c}_t$: Candidato para convertirse en el nuevo estado h_t . Calculado usando x_t y c_{t-1} , utilizando una activación $\tanh$.

3. Segunda componente esencial:

- Γ_t : Controla cuánto de la información pasada (c_{t-1}) y presente (x_t) se utilizará. Calculado con activación sigmoide, generando valores entre 0 y 1.



RNN con GRU

La ecuación clave es:

$$c_t = \Gamma_t \tilde{c}_t + (1 - \Gamma_t) c_{t-1}$$

- Interpretación de Γ_t :

- Si $\Gamma_t \approx 0$, se conserva la información pasada (c_{t-1}).
- Si $\Gamma_t \approx 1$, se actualiza con la nueva entrada ($\tilde{c}_t$).

- Aplicación:

Permite olvidar información irrelevante y centrarse en datos clave para entender frases o secuencias.

◊ Gamma (Γ) “Puerta de Actualización”.

Bidireccionales

- ◊ Información pasada, presente y futura.
- ◊ Acceso a la totalidad de la secuencia.
- ◊ Red hacia adelante (Actual y pasada).
- ◊ Red hacia atrás (Actual y futura)
- ◊ Salida es la combinación lineal de las capas ocultas

